ADBMS Project

Group – 29

ACID Documentation

Project Title: Vehicle Subscription(Rental) Management Platform

Group Members:

Moeed Ali BSCS24136

Abdul Moeed BSCS24140

Transaction Scenario 1: Ride Booking Process

Scenario:

A customer books a ride:

1. Check available driver.
2. Assign driver.
3. Create ride record.
4. Mark driver as unavailable.

Transaction Steps (Logical Flow)

```
START TRANSACTION;

-- 1. Find available driver
SELECT driver_id
FROM Driver
WHERE availability_status = 'Available'
LIMIT 1
FOR UPDATE;

-- 2. Insert Ride
INSERT INTO Ride(customer_id, driver_id, vehicle_id, status, request_time)
VALUES (101, 12, 5, 'Booked', NOW());

-- 3. Update Driver availability
UPDATE Driver
SET availability_status = 'Busy'
WHERE driver_id = 12;

COMMIT;
```

ACID Mapping

I. **Atomicity**
   - ✓ Either ride is created **AND** driver marked busy.
   - ✓ OR nothing happens (ROLLBACK).
   - ✓ IF update fails then entire transaction rolls back.

II. **Consistency**
   - ✓ Foreign key ensures that customer_id, driver_id and vehicle_id exist.
   - ✓ Driver cannot be double booked

III. **Isolation**
   - ✓ Critical Problem:
     - Two customers booking same driver simultaneously.
   - ✓ Solution:

- SELECT . . . FOR UPDATE

IV. Durability

- ✓ After COMMIT, ride record remains even if:
  - MySQL crashes.
  - Server restarts.

Concurrency Strategy: Pessimistic Locking

- ✓ Prevents lost update.
- ✓ Prevents double driver assignment
- ✓ Locks only selected row.

Isolation Level:

- ✓ Prevents dirty reads.
- ✓ Prevents non-repeatable reads.
- ✓ Prevents phantom reads.

Transaction Scenario 2: Ride Completion & Payment Processing

Scenario:

1. Update ride status completed.
2. Calculate fare.
3. Insert payment record.
4. Update driver's total earning.

Transaction Steps:

```
START TRANSACTION;

-- 1. Lock ride
SELECT * FROM Ride
WHERE ride_id = 550
FOR UPDATE;

-- 2. Update ride
UPDATE Ride
SET status = 'Completed',
    end_time = NOW()
WHERE ride_id = 550;

-- 3. Insert payment
INSERT INTO Payment(ride_id, amount, payment_status, payment_date)
VALUES (550, 1250.00, 'Paid', NOW());

-- 4. Update driver earnings
```

```sql
UPDATE Driver
SET total_earnings = total_earnings + 1250.00,
    availability_status = 'Available'
WHERE driver_id = 12;

COMMIT;
```

ACID Mapping

i. **Atomicity**
   - ✓ If payment insertion fails then ride status must not remain Completed.
   - ✓ Everything commits or rollback together.

ii. **Consistency**
   - ✓ Payment must reference valid ride (FK constraint).
   - ✓ Driver earnings always reflect completed rides only.
   - ✓ No partial state allowed.

iii. **Isolation**
   - ✓ Two payments accidentally processed for same ride.
   - ✓ `SELECT . . . FOR UPDATE` Prevents concurrent modifications.

iv. **Durability**
   - ✓ Once committed payments permanently stored.
   - ✓ Earnings permanently updated.

Concurrency Problems Prevented:

- Dirty read.
- Lost update.
- Non-repeatable read.
- Double payment.
- Double driver assignment.

NOT Serializable CAUSE:

- Safest.
- Too slow.
- Creates unnecessary locks.
- Reduces throughput.

As ride system needs **High concurrency** and **Controlled locking**.